

# Source Downgrading: Trust-Bounded Writeback for Derived Memory Records

Diego Falkowski Carboni  
Tyxter  
diego@tyxter.dev

## Abstract

A recursive memory architecture needs more than provenance metadata: it needs an inscription rule under which derived records cannot acquire higher source-integrity than the inputs they were derived from. Without such a rule, an internally reactivated memory can be reasoned over and the reasoning product written back as if externally observed — the failure pattern we call *inference laundering*. Source downgrading is not a new memory type; it is an inscription-integrity invariant for derived memory records. This paper adapts the lattice integrity-meet rule from information-flow control [4, 5, 6] to recursive agent memory, adds an inference-typing ceiling for derivations, binds the rule to a provenance-preserving memory API, and validates the resulting writeback discipline. The algebraic core is standard: a derived value’s integrity cannot exceed the weakest input. The agent-memory contribution is the binding of a source-integrity meet, a derivation-operation ceiling, provenance preservation, and a memory writeback API into one enforceable inscription primitive. This binding yields a concrete laundering failure mode and a paired audit methodology for detecting it.

We compare three policies — naive inscription, provenance propagation alone, and source downgrading — on a deterministic five-case conformance fixture exercising pure inference, simulation contamination, fabrication contamination, chained inference, and mixed chains. Source downgrading achieves zero on inference-laundering rate, derived trust-ceiling violation, false externalization after inference, and provenance recall failure; provenance propagation alone retains a 0.57 ceiling-violation rate because it labels all derived records uniformly as inference. A 20-seed embedding-noise sweep confirms that the deterministic label invariant is independent of retrieval perturbations. A larger 139-session LLM-in-the-loop benchmark evaluates the integrated trace-memory path: contaminated-session laundering falls to 0/111 contaminated sessions versus 36% under vector RAG, while the clean-control set retains 82% decisive action ( $n = 28$ ). As a negative control, PoisonedRAG NQ shows that source downgrading constrains inscription, not retrieval: trace memory reduces attack-success rate from 0.45 to 0.22, but a vector-with-labels control reaches 0.21, indicating that the single-shot RAG gain comes from the source classifier and label-aware prompt, not from the inscription rule. We also identify a methodological constraint: once a label has been laundered, the framework’s own self-audit metrics undercount the failure; truth-grounded ablations remain necessary. We call this property *cascade invisibility*. The central claim is:

derived trust cannot exceed input trust; inference is the upper bound for inscription.

This rule makes recursive self-indexing trust-bounded rather than self-amplifying for architectures that compose memory operations over their own outputs.

**Keywords:** memory, provenance, inference laundering, source monitoring, derived records, trust ordering, recursive memory, self-indexing, agent reliability, LLM agents, information-flow control.

## 1 Introduction

Recursive memory systems compose: the output of memory operations becomes the input of further memory operations. An agent that reasons over a retrieved record produces an inferred claim. The inferred claim may be retained, retrieved again, and reasoned over a second time. Each step composes prior records into derived records. After enough composition, the resulting structure is no longer about what the agent observed; it is about what the agent inferred from what it remembered about what it inferred.

This composition is only useful if it is trust-bounded. A derived record cannot carry higher source integrity than the inputs from which it was derived. If the system silently elevates derived records to the trust level of external observations, the agent enters a regime where its world model is a reinforcement of its own prior inferences. This is exactly the failure pattern that large-language-model agents are observed to exhibit: a fabricated detail is generated, then reasoned over, and the reasoning product is treated downstream as observed evidence [2]. We refer to this pattern as *inference laundering* after the analogous structure in financial systems where the origin of a value is obscured by passing it through intermediate stages.

The flaw is not the absence of provenance metadata. Modern agent memory systems do attach source identifiers to stored records [9, 10, 11]. The flaw is the absence, in the agent-memory API, of an integrity rule that constrains how trust composes when a new record is written from old records. Provenance tells you where a record came from; trust tells you how much weight to give it. A provenance pointer back to a fabricated source does not, by itself, prevent a downstream consumer from treating the pointing record as fact.

The paper therefore treats source downgrading as a writeback invariant: given labeled contributors and a derived write, the memory system computes the permitted source label. The primitive does not decide truth, retrieval, belief, or action.

Throughout the paper, *trust* means source-integrity trust rather than epistemic certainty. Source is how a record was produced; credibility is a revisable belief about the record’s content; confidence is an estimated quality or calibration signal. A confirmed inference may become more credible, but it remains an inference for source-integrity purposes.

This paper instantiates an information-flow-control-style integrity lattice for recursive agent memory and validates it on the inference-laundering failure mode. The result is a static writeback baseline for richer recursive-memory trust mechanisms such as dynamic credibility and cross-agent source transport. The contributions are:

1. A six-class content-source lattice, plus a separate operation-memory class, with a default trust ordering and an operational justification per adjacent pair (§4).

2. A *source-downgrading* inscription primitive that combines the standard lattice integrity meet with an inference-typing rule specific to derived-record semantics: every derived record is capped at the minimum-trust input class, with inference as an absolute upper bound (§5).
3. A demonstration that provenance propagation alone is necessary but not sufficient for safe derived inscription: it preserves chains but does not constrain trust (§5, §7).
4. A methodological finding we call *cascade invisibility*: once a label has been laundered, the system’s own retrospective audit metrics undercount the failure. Truth-grounded ablations remain necessary (§6).
5. Conformance validation on a deterministic five-case fixture, a 20-seed embedding-noise sweep, and a 139-session LLM-in-the-loop benchmark of the integrated source-aware path. We also include a PoisonedRAG negative control to show that the rule is a derived-writeback defence, not a single-shot RAG-poisoning defence (§7).

The paper is positioned narrowly: rather than introducing a new architecture or a new lattice algebra, it applies a known integrity-meet pattern to one inscription decision in recursive memory systems. The motivating case is self-indexed memory: a system whose memory can point back at its own memory must do so under a trust rule, or the indexing structure degenerates into a fixed point of self-reference.

## 2 Threat Model and Non-Goals

Source downgrading is a writeback-time integrity rule. The benchmarks in this paper exercise distinct adversaries, and the rule defends against only one of them by construction. Stating the boundary explicitly prevents the negative-control results of §7 from being misread as refutations.

We distinguish four adversaries that recursive agent memory systems face:

1. **Inference-laundering writeback.** A benign agent reasons over correctly-labeled inputs and writes the derivation back into memory under an inflated source label, after which downstream routing treats the derived record as if externally observed. This is the adversary whose writeback inflation the source-downgrading rule rules out, given correct input labels.
2. **Single-shot RAG poisoning.** An attacker injects passages into the retrieval corpus. The agent retrieves and answers without writing a derived record. The inscription rule is dormant; defence depends on retrieval-time filtering, source classification at ingestion, or label-aware prompting. PoisonedRAG NQ is in this regime.
3. **Source-classifier compromise.** An attacker crafts content designed to induce an inflated source label at the classifier that assigns initial source labels to raw text. Once a record carries an incorrect Source, the inscription rule preserves the incorrect ceiling rather than correcting it. The rule is no stronger than the weakest source-inference decision upstream; we treat classifier robustness as a separate empirical program [3].
4. **Cross-agent label transport.** An attacker (or a careless serialization) strips or upgrades source labels when records move between agents, services, or persistence layers. The receiving system applies source downgrading to inputs whose labels have already been compromised in transit. Defence

requires signed provenance envelopes, source-label translation tables, or verified import policies; these are out of scope here.

The core theorem (Proposition 1) is stated for adversary 1. Its precondition is that contributing records carry authenticated, correctly-assigned source labels and that the declared contributor set is complete with respect to the derivation witness used to produce the derived record. Under that precondition, derived records cannot exceed the weakest input trust, and the inference ceiling holds. The rule does not, on its own, defeat adversaries 2–4. The PoisonedRAG negative control in §7 demonstrates this directly: when no derived record is inscribed, the rule is inactive, and improvements over vanilla retrieval come from the source classifier and label-aware prompt rather than from source downgrading.

Non-goals of the present paper include: (a) defending raw retrieval, ranking, or prompt-time evidence aggregation; (b) detecting adversarial inputs to the source classifier; (c) reconciling conflicting same-rank evidence (this is a routing and belief-revision problem, not an inscription one); (d) securing label transport across API boundaries; (e) calibrating per-record confidence scores; and (f) replacing hallucination detection at generation time. The rule is composable with mechanisms that address those goals, but it is not a substitute for any of them.

### 3 Background and Position

#### 3.1 Motivating fold criterion

The Self-Indexed Mnestic Fold Calculus (SIMFC) defines cognitive memory operationally: a retained record functions as memory only when folding it into the live transition measurably changes a non-bookkeeping transition variable [1]. The paired ablation is

$$\text{fold\_force}(r; s, x) = \|\Phi(s, x, \text{fold}(r)) - \Phi(s, x, \emptyset)\|_{\text{cog}},$$

restricted to non-bookkeeping dimensions of the transition operator  $\Phi$ . A record is memory if and only if its fold-force exceeds an operational threshold on tasks where the record is relevant.

The motivating implementation uses this operational memory criterion, but the source-downgrading rule does not require SIMFC to be accepted as a general theory of memory. It applies to any architecture that writes records derived from prior records back into durable memory. In the implementation studied here, the trust ordering of §4 is applied to records that satisfy the fold criterion rather than to arbitrary text.

#### 3.2 Provenance lineage

Belief-revision systems often preserve lineage: a revised belief is stored with pointers to the prior belief, evidence, update operation, and confidence assigned to the update. The provenance field records how the revision was produced, not only the resulting belief.

The present paper uses that lineage idea for a narrower purpose. Source downgrading preserves provenance — and constrains source integrity — for every derived inscription, including inferences that do not

constitute a belief revision. A derived inference can be a candidate for ordinary trace or operation-memory without ever triggering a revision node; it still needs trust capping.

### 3.3 Routing boundary

Routing decides whether and where a candidate record is written. Source downgrading decides what source label a candidate carries once the system has decided to write it as a derived record. The two decisions are complementary but distinct: routing without a trust rule can pass laundered labels through to durable memory, while a trust rule without routing can produce correctly labeled records that still flow to inappropriate storage. This paper isolates the trust rule.

### 3.4 Information-flow control and integrity lattices

The algebraic core of source downgrading is an integrity-flow rule from information-flow control (IFC), not a new lattice construction. Denning’s lattice model constrains permitted flows among ordered security classes [4]. Later IFC work, including decentralized labels and language-based information-flow security, develops fine-grained labels, declassification, non-interference, and static or dynamic enforcement mechanisms [5, 6]. In confidentiality-oriented IFC, combining data often raises the result’s confidentiality label; in integrity-oriented IFC or taint tracking, the least trustworthy contributor bounds the trustworthiness of the output. The  $\text{Trust}_{\min}$  part of Definition 4 is exactly this meet pattern, specialised to source-integrity labels for agent memory.

The present paper adds three things to that known pattern. First, it defines the source classes used by the recursive-memory implementation and connects them to routing decisions. Second, it adds an inference-typing rule: derivation changes source class even when all inputs are externally observed, so external-plus-external composition yields at most *infer*, not a new *external* record. Third, it binds source-integrity meet, inference typing, transitive provenance, and an *add\_derived*-style memory API into one writeback contract, then validates that contract on a concrete recursive-agent failure mode where derived records are later retrieved as inputs to further derivations. We keep the term *source downgrading* because it names the implementation and benchmark suite, but it should not be confused with IFC declassification or downgrading: the rule here does not authorize release or trust elevation; it caps source labels at writeback.

### 3.5 Database provenance

The provenance component is likewise close to established database-provenance work. Why-, where-, and how-provenance track which source tuples contributed to derived query results and how those contributions combine. Provenance semirings unify several of these accounts through algebraic annotations [7], and provenance surveys distinguish lineage capture from the policies that consume lineage [8]. In this paper,  $\text{Prov}(r)$  is a simple set-valued lineage annotation: it unions contributing record identifiers and their prior provenance. The novelty is not the lineage algebra. It is the binding between lineage, source-integrity meet, inference typing, and an agent-memory inscription API whose downstream routing reads source labels.

Set-valued provenance is intentionally a conservative over-approximation. It records which records contributed but not how they contributed: it does not distinguish central from incidental inputs, does not annotate whether the derivation is a conjunction, disjunction, summary, or contradiction, and does not record whether a downstream verifier invalidated one branch. This is sufficient for a trust-ceiling rule, where the worst-case lineage is what matters, but it is not sufficient for attribution, responsibility assignment, or minimal-witness analysis. A richer provenance algebra — semiring-annotated derivations, contribution weights, or how-provenance polynomials — can be substituted without changing the source-downgrading rule, but the rule itself does not require it. We treat the contribution-sensitive layer as orthogonal future work.

### 3.6 Existing agent memory systems and what they lack

Retrieval-augmented generation [9], MemGPT-style hierarchical memory [10], and Generative Agents [11] all attach source metadata to stored records. Their canonical descriptions do not specify a derived-writeback rule under which new records must inherit a trust ceiling from their inputs. In implementations that write derivations back uniformly, the structural failure mode — a fabricated detail laundered through inference into an externally-trusted record — is therefore reachable unless a separate filtering or inscription policy blocks it.

This is the agent-memory gap addressed here. Source labels exist, but they are not usually tied to an integrity meet at derived writeback time. Provenance pointers exist, but they are not usually bound to a policy that downstream routing must consult.

## 4 The Source Class Lattice

### 4.1 Source classes

We assume the inscription system distinguishes the following source classes for every stored record:

$$\Sigma = \{\text{ext}, \text{tool}, \text{react}, \text{infer}, \text{sim}, \text{fab}, \text{op}\}. \quad (1)$$

The interpretations are:

ext externally observed substrate-state (sensor input, observed environment event);

tool tool-output substrate-state (output of an executed tool or external API);

react reactivated prior memory (content retrieved from the agent’s own store);

infer inferred substrate-state (model-derived content, including chain-of-thought conclusions);

sim simulated or hypothetical substrate-state (counterfactual or imagined content explicitly tagged as such);

fab fabricated or source-uncertain substrate-state (content of unknown provenance, including model hallucinations);

op operation-memory record (record of how memory use changed a transition).

These classes are not exhaustive of all possible memory ontologies; they are the minimum set that distin-

guishes inscription decisions an agent system must make in our experiments. Implementations may refine within a class without changing the rule of §5.

## 4.2 Trust ordering

We define a total order on  $\Sigma \setminus \{\text{op}\}$  (the operation class is treated separately because it indexes events rather than content):

**Definition 1** (Trust ordering). *A trust function  $\text{Trust} : \Sigma \setminus \{\text{op}\} \rightarrow \mathbb{N}$  assigns integer ranks:*

$$\begin{aligned} \text{Trust}(\text{fab}) &= 0, & \text{Trust}(\text{sim}) &= 1, & \text{Trust}(\text{infer}) &= 2, \\ \text{Trust}(\text{react}) &= 3, & \text{Trust}(\text{tool}) &= 4, & \text{Trust}(\text{ext}) &= 5. \end{aligned}$$

We say  $s$  is at least as trusted as  $s'$ , written  $s \succeq s'$ , iff  $\text{Trust}(s) \geq \text{Trust}(s')$ .

The ordering is axiomatic. We motivate it operationally pair-by-pair:

1.  $\text{fab} \prec \text{sim}$ . A fabricated record has unknown provenance; a simulated record has explicit hypothetical status and an internally consistent generation context. Adjudicating between them, simulated records are at least as informative because their context is recoverable.
2.  $\text{sim} \prec \text{infer}$ . A simulated record is explicitly counterfactual; an inferred record is a derivation from records the system actually holds. Inferences are constrained by the inputs that generated them; simulations are not.
3.  $\text{infer} \prec \text{react}$ . A reactivated record was previously held by the system, presumably for a reason. An inferred record may have been generated fresh. Reactivation is evidence that a record passed earlier inscription decisions; pure inference carries no such history.
4.  $\text{react} \prec \text{tool}$ . A tool-output record is an externally produced artifact (the tool ran and returned a value); a reactivated record is an internal copy. Tool outputs introduce information from outside the agent's mnestic state; reactivation merely surfaces what was already there.
5.  $\text{tool} \prec \text{ext}$ . A tool output depends on the tool's correctness and the agent's framing of the tool call. An external observation is constrained only by the substrate. External observations are the boundary condition for everything else; without them, the agent has no anchor.

This ordering is a default engineering policy for the implementation tested in §7, not a universal reliability ontology. In particular,  $\text{react} \prec \text{tool}$  is appropriate when tool output is treated as an external artifact and reactivation as an internal copy. Other deployments may invert that operational judgment: a reactivated record with signed provenance can be more reliable than a tool output from an adversarial or unaudited channel. Such systems should attach per-record reliability metadata, signatures, or verification qualifiers alongside the source class. The source-downgrading rule then remains the conservative default over source labels, while routing can consult the additional reliability layer.

Production systems may refine this default with per-record reliability or context-sensitive policy. The formal development below fixes a single global order so that the inscription invariant can be stated and tested cleanly.

The motivation is not that this ordering matches any specific cognitive theory of source monitoring [12, 13]. It is that the ordering is sufficient for the inscription rule of §5 on the fixture of §7. Alternative adjacent rankings are deployment policy choices and should be validated against the same gates before use.

### 4.3 The min-trust function

**Definition 2** (Min-trust of a set of inputs). *For a non-empty set of input source labels  $S = \{s_1, \dots, s_k\}$  with each  $s_i \in \Sigma \setminus \{\text{op}\}$ , the min-trust source is*

$$\text{Trust}_{\min}(S) = \arg \min_{s_i \in S} \text{Trust}(s_i).$$

*When  $S$  contains classes of equal minimum trust,  $\text{Trust}_{\min}(S)$  returns any of them (the choice does not affect downstream routing because trust rank is what is consulted).*

**Definition 3** (Inference ceiling). *For any non-empty set of contributing input labels  $S$ , the inference-ceiling source is*

$$\text{Trust}_{\text{ceil}}(S) = \begin{cases} \text{Trust}_{\min}(S), & \text{if } \text{Trust}(\text{Trust}_{\min}(S)) \leq \text{Trust}(\text{infer}), \\ \text{infer}, & \text{otherwise.} \end{cases}$$

The inference ceiling captures the operational claim that even when all inputs are externally observed, the act of deriving a new claim from them produces at most an inferred record. External-from-external composition does not yield a new external observation; it yields an inference. The ceiling is the upper bound; the floor is the minimum-trust input.

### 4.4 Origin labels, operation labels, and the collapsed source

The lattice as presented mixes two conceptually different things, and it is worth surfacing the distinction explicitly. Each record carries information of two kinds:

**Origin label** *where* the content entered the system. The natural origin classes are *ext* (boundary observation), *tool* (executed tool output), *react* (already in the agent’s store), and the absence of origin for internally generated content.

**Operation label** *how* the record was produced. The natural operation classes are *derived* (output of an inference step), *simulated* (output of a hypothetical generation step), *fabricated* (output of a generation step with unverifiable provenance), and *observed* (no generation step occurred).

The single Source label of §4 is a conservative collapse of these two dimensions onto one trust ordering. *ext*, *tool*, and *react* are origin labels in this collapse, applicable when no derivation operation has been applied to the content. *infer*, *sim*, and *fab* are operation labels in this collapse, applicable when a derivation operation has been applied and the operation determines the integrity ceiling regardless of input origin.

Under this reading, the inference ceiling of Definition 3 is not a separate rule but a consequence: derivation is an operation, and the operation imposes *infer* as the operation-typing label even when all input origins were *ext*. The collapsed lattice is then internally consistent:  $\text{Trust}_{\text{ceil}}(S)$  first looks at the operation



that produced  $r$  (if any), then at the minimum origin trust among inputs, and returns the lower of the two. The implementation tested in §7 computes the same thing because derivation is the only operation that produces a new record from old records; in a richer system with multiple operation classes (summarization vs. entailment vs. counterfactual generation), each operation supplies its own ceiling and the rule is applied as  $\text{Source}(r) = \min(\text{operation-ceiling}, \text{Trust}_{\min}(\text{origins}))$ .

We keep the single collapsed label in the formal development because it is sufficient for the inscription invariant and matches the API surface of existing memory systems. The origin/operation factoring is the natural extension when an implementation needs to track *why* a derived record was capped — whether by a low-trust input or by the operation type itself — for routing or audit purposes.

## 5 The Source-Downgrading Primitive

### 5.1 The inscription rule

**Definition 4** (Source-downgrading inscription). *Let  $r$  be a derived record constructed from a non-empty set of contributing input records  $C = \{c_1, \dots, c_k\}$  with source labels  $S = \{\text{Source}(c_i)\}$  and provenance fields  $\{\text{Prov}(c_i)\}$ . Source-downgrading inscription writes  $r$  with:*

$$\text{Source}(r) = \text{Trust}_{\text{ceil}}(S), \quad (2)$$

$$\text{Prov}(r) = \bigcup_{c_i \in C} (\text{Prov}(c_i) \cup \{\text{id}(c_i)\}). \quad (3)$$

*Equivalently, the derived source is capped at the lowest-trust contributing input (or at infer if all inputs are above it), and the derived provenance is the transitive closure of contributing provenance plus contributing record identifiers.*

The definition has a support precondition:  $C$  must be complete with respect to the derivation witness used to produce  $r$ . Every record whose content materially supports the derived claim must appear either in  $C$  or in the stored provenance closure of an element of  $C$ . If a derivation uses a simulated record but the caller declares only an external record, the caller has omitted a contributor and the computed ceiling is no longer sound. This moves the laundering attack surface from caller-supplied labels to caller-supplied incomplete witnesses; practical enforcement requires sealed generation contexts, automatic derivation tracing, append-only derivation logs, support witnesses, entailment checks, or audit hooks.

Operational edge cases are part of the API contract. An empty contributing set is not a derivation; implementations must reject it or label the result *fab*. In the model above,  $\text{Prov}(r)$  stores a finite transitive closure over contributor identifiers and origin tokens. An edge-stored implementation may instead store only direct contributors, but any routing or audit decision that relies on provenance must compute the same closure before use. Cycles must either be rejected at write/import time or traversed with a visited set. Same-rank contradictions, such as two disagreeing tool outputs, are not resolved by source downgrading and should raise a separate contradiction flag for routing or belief revision. Hybrid-source content should be split into smaller records before inscription. Finally, even complete contributors may be semantically irrelevant to the

derived claim; validating that support relation requires a derivation witness, entailment check, or audit hook outside the source-label rule itself.

Two structural facts follow:

**Proposition 1** (Trust monotonicity). *Under source-downgrading inscription,  $\text{Trust}(\text{Source}(r)) \leq \min_i \text{Trust}(\text{Source}(c_i))$  and  $\text{Trust}(\text{Source}(r)) \leq \text{Trust}(\text{infer})$ .*

*Proof.* Immediate from Definitions 2 and 3. □

**Proposition 2** (Transitive provenance closure). *Under source-downgrading inscription, every provenance origin stored in any contributing input’s provenance closure is stored in the derived record’s provenance closure, and every direct contributor identifier is stored as well.*

*Proof.* Immediate from Definition 4:  $\text{Prov}(r)$  is the union, over all contributors, of each stored contributor closure  $\text{Prov}(c_i)$  and each direct contributor, identified as  $\text{id}(c_i)$ . Thus no origin already present in a contributor’s closure is dropped when  $r$  is written. If an implementation stores only direct provenance edges, the same statement is obtained by taking the finite transitive closure of those edges before routing or audit. □

## 5.2 Why provenance propagation alone is insufficient

A weaker policy — which we call *provenance propagation* — preserves  $\text{Prov}(r)$  as in Definition 4 but always sets  $\text{Source}(r) = \text{infer}$  regardless of input trust. Provenance is preserved but the trust ceiling is not enforced.

**Property 1** (Failure of pure provenance propagation). *There exist input sets  $S$  and downstream routing policies under which pure provenance propagation yields derived records that route to trusted write targets even though some  $c_i \in C$  has  $\text{Source}(c_i) \in \{\text{sim}, \text{fab}\}$ .*

*Sketch.* Suppose  $C$  contains one external record and one simulated record. Pure provenance propagation labels  $r$  as *infer*. A routing policy that promotes *infer* records to operation-memory or durable memory will then route  $r$  to a trusted target. The simulation-derived contribution is preserved in  $\text{Prov}(r)$  but is not consulted by the routing decision, which reads  $\text{Source}(r) = \text{infer}$ . We demonstrate this empirically in §7, where pure provenance propagation produces a 0.57 ceiling-violation rate on the fixture. □

## 5.3 Why naive inscription cannot be the default

The default of many memory APIs is what we call *naive inscription*: a derived record is written with whatever default source label the API supplies (often *external*), and provenance is not propagated unless the caller explicitly sets it. Naive inscription is the laundering baseline.

**Property 2** (Naive inscription launders by default). *Under naive inscription with default source external, every derived record violates the trust ceiling for any input set  $S$  containing at least one non-external record, and additionally for any input set  $S$  in which all inputs are external (because the inference ceiling rejects an external label on a derived record).*

This is what makes naive inscription dangerous as a default: not that it launders only sometimes, but that it launders always. The empirical result of §7 confirms this — naive inscription produces a 1.0 trust-ceiling violation rate on every seed of the multi-seed sweep.

#### 5.4 Worked example: depth-3 derivation chain

To make the API contract concrete, we trace one call to `add_derived` through a chain in which the contamination is introduced at the second hop. The records and contributing sets are:

- $E_1$ : original observation.  $\text{Source}(E_1) = \text{ext}$ .  $\text{Prov}(E_1) = \emptyset$ .
- $T_1$ : tool output.  $\text{Source}(T_1) = \text{tool}$ .  $\text{Prov}(T_1) = \emptyset$ .
- $D_1$ : derived from  $\{E_1, T_1\}$  by an inference step.
- $H_1$ : simulated hypothesis injected into the chain.  $\text{Source}(H_1) = \text{sim}$ .  $\text{Prov}(H_1) = \emptyset$ .
- $D_2$ : derived from  $\{D_1, H_1\}$  by an inference step.
- $D_3$ : derived from  $\{D_2\}$  by a second inference step.

The inscription system computes each derived record by Definition 4. We walk through each call.

**Step 1:  $D_1$  from  $\{E_1, T_1\}$ .** The contributing label set is  $S_1 = \{\text{ext}, \text{tool}\}$ , with ranks  $\text{Trust}(\text{ext}) = 5$  and  $\text{Trust}(\text{tool}) = 4$ . The minimum-trust input is  $\text{Trust}_{\min}(S_1) = \text{tool}$  at rank 4, which exceeds  $\text{Trust}(\text{infer}) = 2$ , so the inference ceiling triggers:  $\text{Trust}_{\text{ceil}}(S_1) = \text{infer}$ . Provenance is the union of contributor identifiers:  $\text{Prov}(D_1) = \{E_1, T_1\}$ . The system writes  $D_1$  with  $\text{Source}(D_1) = \text{infer}$  and  $\text{Prov}(D_1) = \{E_1, T_1\}$ . Even with two non-contaminated, externally-anchored inputs, the derived record is capped at *infer*; this is the inference ceiling in action.

**Step 2:  $D_2$  from  $\{D_1, H_1\}$ .** The contributing label set is  $S_2 = \{\text{infer}, \text{sim}\}$ . The minimum-trust input is  $\text{Trust}_{\min}(S_2) = \text{sim}$  (rank 1). The minimum is at or below  $\text{Trust}(\text{infer}) = 2$ , so the inference ceiling does *not* override:  $\text{Trust}_{\text{ceil}}(S_2) = \text{sim}$ . Provenance accumulates transitively:  $\text{Prov}(D_2) = \text{Prov}(D_1) \cup \{D_1\} \cup \text{Prov}(H_1) \cup \{H_1\} = \{E_1, T_1, D_1, H_1\}$ . The system writes  $D_2$  with  $\text{Source}(D_2) = \text{sim}$ ,  $\text{Prov}(D_2) = \{E_1, T_1, D_1, H_1\}$ . The contamination from  $H_1$  has been recorded both in the source ceiling and in the provenance closure.

**Step 3:  $D_3$  from  $\{D_2\}$ .** The contributing label set is  $S_3 = \{\text{sim}\}$ . The minimum-trust input is  $\text{sim}$ . The inference ceiling does not override.  $\text{Trust}_{\text{ceil}}(S_3) = \text{sim}$ .  $\text{Prov}(D_3) = \text{Prov}(D_2) \cup \{D_2\} = \{E_1, T_1, D_1, H_1, D_2\}$ . The system writes  $D_3$  with  $\text{Source}(D_3) = \text{sim}$ ,  $\text{Prov}(D_3) = \{E_1, T_1, D_1, H_1, D_2\}$ .

**Observations.** The ceiling is monotone non-increasing along the chain:  $\text{ext}, \text{tool} \mapsto \text{infer} \mapsto \text{sim} \mapsto \text{sim}$ . Once a low-trust contributor enters the chain, the ceiling cannot recover, and the contamination is visible at every subsequent step both in `Source` and in `Prov`. A downstream routing decision over  $D_3$  that consults `Source` alone will see *sim* and route to quarantine; a decision that walks `Prov` will additionally find  $H_1$  as a simulation contributor. Under naive inscription, by contrast, each derived record would be written with the default *ext* label,  $H_1$ 's contribution would never be reflected in any subsequent record, and routing would

treat  $D_3$  as externally observed. This worked example is what the `adversarial_reload_v2` benchmark of §7 exercises in the depth-3 condition with LLM-generated text rather than fixed-content records.

## 6 Cascade Invisibility

A separate methodological finding emerges from the harness: once a derived record has been laundered, the failure becomes harder for the framework itself to detect on subsequent steps. This is because the most natural "self-audit" metric for a memory system is to inspect the source labels of contributing records when a new derivation is written and count the cases in which the new record's label is inconsistent with its inputs.

Define the local audit metric as follows:

**Definition 5** (Local laundering rate). *The local laundering rate of a policy on a derivation step is*

$$\text{LR}_{\text{local}}(\text{step}) = \mathbb{K}[\exists c_i \in C : \text{Source}(c_i) \neq \text{ext} \wedge \text{Source}(r) = \text{ext}],$$

*that is, an indicator that some input was non-external and the derived record was labeled external.*

**Definition 6** (Truth-grounded laundering rate). *Suppose the fixture supplies a per-case expected maximum trust  $\text{Trust}_{\text{max}}^*(\text{case})$  for the final derived record, computed from the contributing source labels at fixture-generation time. The truth-grounded violation rate is*

$$\text{LR}_{\text{truth}}(\text{step}) = \mathbb{K}[\text{Trust}(\text{Source}(r)) > \text{Trust}(\text{Trust}_{\text{max}}^*(\text{case}))],$$

*that is, whether the derived record exceeds the fixture's expected ceiling.*

For a single-step derivation,  $\text{LR}_{\text{local}}$  and  $\text{LR}_{\text{truth}}$  agree on the laundering verdict. They diverge on chained derivations:

**Property 3** (Cascade invisibility). *There exist fixtures in which  $\text{LR}_{\text{local}}$  strictly undercounts  $\text{LR}_{\text{truth}}$  for chained derivations under a laundering policy. More specifically, consider a linear derivation chain in which each derived record has exactly one contributor, each laundering step writes the derived record as external, and the local metric tests only whether a non-external input was rewritten as external. In that restricted setting, the undercount equals the number of chained derivations whose contributing records' labels were already laundered to external on prior steps.*

*Sketch.* Consider a chain  $E_1 \xrightarrow{\text{infer}} D_1 \xrightarrow{\text{infer}} D_2$  where  $E_1$  is external. Under naive inscription,  $D_1$  is labeled *external*. When  $D_2$  is written from  $D_1$ , the local audit reads  $\text{Source}(D_1) = \text{ext}$  and finds no non-*external* contributor, so  $\text{LR}_{\text{local}}(D_2) = 0$ . But the fixture's  $\text{Trust}_{\text{max}}^*$  for  $D_2$  is at most *infer* (two derivation steps from observation), so  $\text{LR}_{\text{truth}}(D_2) = 1$ . In a linear chain satisfying the stated hypotheses, the same argument applies to every subsequent step: the local audit sees only the already-laundered *external* label, while the truth-grounded ceiling remains at most *infer* or lower. Summing over the chain gives the stated undercount. The framework cannot recover this information from its own internal labels.  $\square$

**Prediction 1** (Audit protocol requirement). *Any audit of a memory architecture for inference-laundering safety must compare two metrics: a self-referential metric computed from stored source labels, and a truth-grounded metric computed against fixture- or out-of-system labels. A single-metric audit using only the self-referential metric will report a false floor on the laundering rate.*

In production, the audit anchor is rarely truth in the omniscient sense; it is an *external anchor* or *out-of-band* check that is not derived from the same labels the system is auditing. Practical anchors include immutable append-only provenance logs, canary contaminants, sampled oracle review, independent shadow source classifiers, signed source envelopes, and claim-level support verification. The correct anchor depends on the threat model. The methodological requirement is that at least one audit path not depend solely on the labels currently stored in the memory system being audited. We use the term *truth-grounded* for the fixture-side metric (Definition 6), where ground truth is supplied by construction; for production deployments, *external-anchor audit* is the more honest term.

This is the methodological constraint cascade invisibility imposes. It is a property of memory architectures in general, not of any specific implementation. We empirically demonstrate it on our fixture in §7: under naive inscription,  $LR_{\text{local}}$  reports 0.43 across the 7 derived records of the fixture while  $LR_{\text{truth}}$  reports 1.0, a gap of 0.57 that is structurally unrecoverable from internal labels alone.

## 7 Validation Harness

### 7.1 Five-case conformance fixture

The deterministic conformance fixture, implemented in `src/fgm/laundering.py`, contains five cases that together exercise the laundering chain across the relevant input combinations:

1. **Pure inference from observations** ( $D_{\text{pure}}$  from two external records). Expected ceiling: *infer*.
2. **Laundering from simulation** ( $D_{\text{sim}}$  from one external and one simulated record). Expected ceiling: *sim*.
3. **Laundering from fabrication** ( $D_{\text{fab}}$  from one external and one fabricated record). Expected ceiling: *fab*.
4. **Chained inference from observation** ( $D_{\text{step2}}$  from  $D_{\text{step1}}$  from one external record). Expected ceiling: *infer*; provenance must walk back to the original observation.
5. **Mixed chain simulation then inference** ( $D_{\text{mix2}}$  from  $D_{\text{mix1}}$  from external and simulated records). Expected ceiling: *sim*.

The five cases produce seven derived records in total. Each case includes a later query about the derived claim and an expected routing verdict: cases 1 and 4 expect later queries to route to a trusted target (the chain is honest); cases 2, 3, and 5 expect the later queries to route to quarantine.

### 7.2 Three policies

Three inscription policies are compared:

**naive\_inscribe** uses the default arguments of `agent.add(content)`: source defaults to *external*, provenance is empty. This is the laundering baseline.

**provenance\_propagating** sets  $\text{Source}(r) = \text{infer}$  and propagates provenance as in Definition 4, but does not cap trust at the input minimum.

**source\_downgrading** applies Definition 4 in full: source is  $\text{Trust}_{\text{ceil}}(S)$  and provenance is the transitive closure.

### 7.3 Metrics

Five metrics are reported per policy:

***Inference laundering rate*** ( $\text{LR}_{\text{local}}$ ) Definition 5, averaged over derived records.

***Provenance chain recall*** fraction of derived records whose transitive provenance closure contains the fixture’s `expected_provenance_origins`.

***False externalization after inference*** fraction of later queries whose target derived record was retrieved and routed to a trusted write target when the fixture specified that the chain should be quarantined.

***Derived trust-ceiling violation rate*** ( $\text{LR}_{\text{truth}}$ ) Definition 6, averaged over derived records.

***Transitive provenance depth mean*** mean walk-back depth of provenance pointers from derived records to original observation tokens.

### 7.4 Single-seed results

Table 1 shows the single-seed (seed=0, canonical hash embeddings) results.

| Policy                 | $\text{LR}_{\text{local}}$ | Prov. recall | False-ext. | $\text{LR}_{\text{truth}}$ | Prov. depth |
|------------------------|----------------------------|--------------|------------|----------------------------|-------------|
| naive_inscribe         | 0.43                       | 0.00         | 0.40       | 1.00                       | 0.00        |
| provenance_propagating | 0.00                       | 1.00         | 0.40       | 0.57                       | 2.29        |
| source_downgrading     | 0.00                       | 1.00         | 0.00       | 0.00                       | 2.29        |

Table 1: Single-seed conformance results on the laundering fixture (5 cases, 7 derived records). Source downgrading achieves zero on all four failure metrics. Provenance propagation alone eliminates local laundering and recovers full provenance, but cannot prevent trust-ceiling violations (it always labels derived records as *infer*, which over-trusts simulation- and fabrication-derived inferences).

Two findings are worth stating explicitly. First, the gap between  $\text{LR}_{\text{local}} = 0.43$  and  $\text{LR}_{\text{truth}} = 1.00$  under `naive_inscribe` is the cascade-invisibility property of §6: the framework’s own self-audit reports a 43% laundering rate while the truth-grounded audit reports 100%. Second, the  $\text{LR}_{\text{truth}} = 0.57$  under `provenance_propagating` corresponds exactly to the four-out-of-seven derived records that originate from simulation or fabrication inputs (where the expected ceiling is below *infer*). Provenance preservation cannot rescue these cases because routing reads the label, not the chain.

## 7.5 Multi-seed robustness

To probe whether the result is structural rather than retrieval-contingent, we run the harness across  $N = 20$  seeds where seed 0 uses canonical hash embeddings and seeds 1–19 apply reproducible Gaussian noise ( $\sigma = 0.05$ ) to embeddings. Each seed–text pair receives deterministic noise, so the multi-seed run is itself reproducible.

Table 2 reports the mean  $\pm$  standard deviation across seeds.

| Policy                 | LR <sub>local</sub> | False-ext.        | LR <sub>truth</sub> |
|------------------------|---------------------|-------------------|---------------------|
| naive_inscribe         | $0.429 \pm 0.000$   | $0.520 \pm 0.117$ | $1.000 \pm 0.000$   |
| provenance_propagating | $0.000 \pm 0.000$   | $0.520 \pm 0.117$ | $0.571 \pm 0.000$   |
| source_downgrading     | $0.000 \pm 0.000$   | $0.000 \pm 0.000$ | $0.000 \pm 0.000$   |

Table 2: Multi-seed ( $N = 20$ ) results under embedding-noise perturbation ( $\sigma = 0.05$ ). Most metrics show zero variance because they are determined by inscription-policy decisions (which read source labels) rather than retrieval ranking. The only retrieval-dependent metric is false externalization after inference: it varies with seed for the laundering policies because later queries occasionally fail to retrieve the derived record at all. Under source downgrading the metric remains zero across all seeds, because the derived record is correctly labeled and the routing layer quarantines it regardless of retrieval order.

The structural property is now visible. Under source downgrading, every failure metric is 0 with 0 variance across the noise distribution. Under the laundering policies, the truth-grounded metrics are stable (laundering depends on labels, not on retrieval), but the false-externalization metric varies with retrieval-order perturbation because the later query’s retrieval result is itself noise-sensitive. The retrieval noise determines whether the laundered record reaches the routing decision; the labeling determines what routing decides if reached. In this harness, the source-downgrading label invariant is not an artifact of retrieval order.

## 7.6 Extended adversarial fixture: DAG structure and rule boundaries

The five-case conformance fixture exercises linear contamination chains. To probe DAG-shaped derivation patterns and the boundaries the rule explicitly does not address, we add a smaller *extended adversarial fixture* of five additional cases. Each case is hand-constructed to test exactly one structural pattern; results are reported as per-record assertions rather than as case-level aggregates, because the per-case `expected_max_trust` field is too coarse for cases with intermediate records that legitimately carry higher trust than the final.

The cases are implemented in `make_extended_adversarial_fixture()` alongside the original fixture, and the per-record assertions live in `tests/architecture/test_laundering.py`. Under source downgrading, the rule succeeds on the three pattern-extension cases as expected, behaves correctly within scope on the two scope-limit cases, and fails by design on the one transport-boundary case:

**Branching DAG** one contaminated seed fans out to three parallel derivations. All three derived records correctly inherit the *sim* ceiling; provenance contains both contributing seeds. The rule applies pointwise to fan-out.

**Reconvergent DAG** a clean and a contaminated branch merge at a later step. The intermediate clean-branch record gets *infer*; the intermediate contaminated-branch record gets *sim*; the final reconverged record gets *sim*. Provenance on the final record reaches all three seed origins through both paths, with the visited-set traversal correctly deduplicating reconvergent ancestors.

**Same-rank contradiction** two *tool*-rank contributors disagree. The derived record gets *infer* (the inference ceiling triggers because *tool* is above it), and both contributors remain in provenance. The contradiction itself is not flagged; the rule preserves contributors for downstream contradiction-handling logic but does not adjudicate same-rank conflicts. This matches the scope statement of §5.

**Irrelevant declared contributor** a semantically unrelated input is declared as a contributor. The derived record gets *infer* and the irrelevant contributor remains in provenance. The rule has no mechanism to detect the lack of support; validating the support relation requires a derivation witness or entailment check outside the source-label layer, again as stated in §5.

**Cross-agent label stripped** a simulated input is planted with its label stripped to *ext* (modelling the cross-agent transport boundary, adversary 4 of §2). All inputs now appear external, the inference ceiling triggers, and the derived record gets *infer*. The truth-grounded ceiling is *sim*, so the rule over-trusts the derivation by one rank. This is the single case in the extended fixture where source downgrading fails, and it fails for exactly the reason §2 states: the rule is no stronger than the weakest source-inference decision upstream.

The pattern that emerges is the one the paper claims: four cases inside the rule’s scope are handled correctly (three by direct rule application, two of those by deliberately not over-acting on out-of-scope contradiction or irrelevance), and one case outside the rule’s scope (label transport across the system boundary) fails by design. This is a small but honest extension of the empirical claim: the rule does what the threat model says, and not more.

## 7.7 LLM-in-the-loop adversarial reload v2

The deterministic conformance fixture tests the inscription invariant directly. To test the full system under free-text LLM derivations, we also ran the `adversarial_reload_v2` benchmark in the `trace-memory` repository. The benchmark contains 139 multi-turn sessions across seven domains: SRE, customer support, security, finance, healthcare, legal, and research. Each session contains one seed observation, one auxiliary observation that is either fabricated, simulated, or clean, a chain of one to three LLM-generated derivations, and a reload question that retrieves the derived record and asks for an action. There are 111 contaminated sessions and 28 clean controls. Input labels are authored oracle labels; this benchmark tests system behavior given correct labels, not the classifier that assigns labels to raw text.

This benchmark evaluates the integrated writeback-and-use path. The `trace_memory` arm combines source-downgraded writeback, label-preserving retrieval, routing choices, and a label-aware answer prompt; the conformance harness above isolates the inscription rule itself.



### 7.7.1 Arms and architectures

Four arms are compared:

**vector** standard retrieval-augmented generation. Records are stored as embeddings with source-label metadata that the answer prompt ignores. Derived records are written without a trust ceiling.

**trace\_memory** the source-downgrading inscription rule of §5 bound to the trace-memory API. Derived records inherit  $\text{Trust}_{\text{ceil}}$  over input labels; retrieval preserves labels and the answer prompt is label-aware.

**bash** a tool-use arm in which the agent executes shell commands to inspect a labeled record store. Source labels are visible in retrieved content.

**bash\_nolabels** identical to bash but with source labels stripped from retrieved content, isolating the contribution of label visibility from the tool-use modality.

### 7.7.2 Metrics

Each session is graded by a deterministic substring rubric that maps the agent’s parsed final answer to one of {safe, unsafe, quarantine, fallback}. Five quantities are reported per arm:

**Strict** fraction of sessions where the parsed answer equals the dataset’s literal `expected_answer_id`. This is the original benchmark rubric.

**Defensible** strict-correct or (parsed answer is `quarantine` and the session is contaminated). This credits architecturally-correct conservative behaviour: abstaining is a principled choice when contaminated memory is present, even when the dataset’s literal expected answer was `safe`.

**Unsafe** fraction of sessions where the parsed answer is in the case’s `unsafe_answer_ids` set, that is, an answer the dataset marks as actively harmful given the scenario.

**Contam-unsafe** Unsafe restricted to the  $n = 111$  contaminated sessions: how often the arm took an actively unsafe action given contaminated memory was present. This is the architectural failure-mode metric.

**Clean-act** strict-correct rate restricted to the  $n = 28$  clean controls: how often the arm took the decisive action on a clean session, where decisive action is the dataset’s strict-correct answer. Lower values flag over-quarantine.

Definitions follow the implementation in `product_comparison/grading.py`. A session is classified as contaminated if at least one of its memories has source label *sim* or *fab*, and as a clean control otherwise.

### 7.7.3 Judge methodology

Table 3 reports the substring-grader numbers because they are deterministic and reproducible. We additionally cross-check the substring grades with a constrained-rubric LLM judge (gpt-4.1-mini, JSON-schema reply, judge code in `product_comparison/judge.py`). The judge receives the reload question, the contamination flag, and the agent’s raw final reply, and returns three booleans:

**recognized\_contamination** true if the response identifies the contaminating memory as unverified, hypothetical, simulated, or not actionable evidence; vacuously true on clean sessions.

***recommended\_action*** true if the response recommends taking the question’s prompted action (deploying, escalating, paying, etc.); false on quarantine, abstention, or request-for-evidence responses.

***defensibly\_safe*** true if recommending action is defensible given the available evidence. For contaminated scenarios, recommending action is not defensibly safe; for clean scenarios, recommending action is defensibly safe.

The full judge system prompt is reproduced verbatim in the code reference. The judge plays the role of a semantic cross-check on the substring grader, not a replacement: when the two disagree, we manually inspect the case rather than trust either rubric in isolation. On the run reported here, the substring grader’s *Contam-unsafe* numbers and the judge’s  $\neg$ *defensibly\_safe* rate are within one session of each other for all four arms, so we report the substring numbers throughout.

#### 7.7.4 Aggregate results

| Arm                  | Strict | Defensible | Unsafe | Contam-unsafe        | Clean-act            |
|----------------------|--------|------------|--------|----------------------|----------------------|
| <b>vector</b>        | 0.237  | 0.612      | 0.288  | 0.360 [0.277, 0.453] | 0.500 [0.326, 0.674] |
| <b>trace_memory</b>  | 0.216  | 0.964      | 0.000  | 0.000 [0.000, 0.033] | 0.821 [0.644, 0.921] |
| <b>bash</b>          | 0.755  | 0.813      | 0.000  | 0.000 [0.000, 0.033] | 0.071 [0.020, 0.226] |
| <b>bash_nolabels</b> | 0.763  | 0.799      | 0.000  | 0.000 [0.000, 0.033] | 0.071 [0.020, 0.226] |

Table 3: Adversarial-reload v2 results over 139 LLM-in-the-loop sessions using `gpt-4.1-mini`. Brackets give Wilson 95% confidence intervals; *Contam-unsafe* has  $n = 111$  and *Clean-act* has  $n = 28$ . *Contam-unsafe* is the architectural failure-mode metric: the rate at which an arm acts on contaminated derivation chains. *Clean-act* measures decisive action on clean controls; the wide interval at  $n = 28$  should be read as an initial over-quarantine check, not a full utility evaluation.

The result is aligned with the deterministic conformance fixture. Vector RAG launders contamination on 36% of contaminated sessions (95% CI [0.28, 0.45]) because the derived text is stored and retrieved without a source-integrity ceiling visible to the response policy. The three label-aware arms register zero contaminated-session unsafe answers, with a Wilson upper bound of 0.033 at  $n = 111$ ; the three CIs are non-overlapping with vector’s. On clean controls, the `trace_memory` arm retains an 82% decisive-action rate (95% CI [0.64, 0.92]); the two `bash` arms are pinned at 7% (95% CI [0.02, 0.23]), indicating heavy over-quarantine under shell-based retrieval regardless of label visibility. The clean-control interval at  $n = 28$  is wide, so this is an over-quarantine check rather than a full utility evaluation.

#### 7.7.5 Breakdown by contamination type and chain depth

The vector-RAG failures are not uniformly distributed across the 111 contaminated sessions. Table 4 reports *Contam-unsafe* for each arm sliced by contamination type (fabricated vs. simulation) and by chain depth (one to three derivation hops).

| Arm           | Fabricated       |                 |                 | Simulation       |                 |                 |
|---------------|------------------|-----------------|-----------------|------------------|-----------------|-----------------|
|               | $d=1$ ( $n=46$ ) | $d=2$ ( $n=7$ ) | $d=3$ ( $n=4$ ) | $d=1$ ( $n=44$ ) | $d=2$ ( $n=7$ ) | $d=3$ ( $n=3$ ) |
| vector        | 0.043            | 0.143           | 0.000           | 0.705            | 0.571           | 0.667           |
| trace_memory  | 0.000            | 0.000           | 0.000           | 0.000            | 0.000           | 0.000           |
| bash          | 0.000            | 0.000           | 0.000           | 0.000            | 0.000           | 0.000           |
| bash_nolabels | 0.000            | 0.000           | 0.000           | 0.000            | 0.000           | 0.000           |

Table 4: *Contam-unsafe* by contamination type and chain depth. Vector RAG’s failures concentrate in simulation contamination ( $\approx 70\%$ ) and are mild on fabricated contamination ( $\approx 5\%$ ), suggesting that the underlying model recognises blatantly fabricated content but does not treat clearly-labeled hypotheticals as non-evidence. The three label-aware arms register zero failures in every cell.

This breakdown is consistent with the failure mode isolated by the conformance harness. Without source labels reaching the answer prompt, the underlying LLM is mostly able to discount blatantly fabricated content on its own — the 4% to 14% fabricated-contamination unsafe rates for vector are not catastrophic — but it readily acts on labeled simulations, with 57% to 70% unsafe rates across depths. The inscription rule does not need to teach the model that fabrications are dangerous; it needs to surface the label so that a simulated hypothesis is treated as not-evidence rather than as evidence. The depth structure also indicates that contamination at deeper chains is harder, not easier, to launder past vector retrieval (the simulation rate is approximately flat across depths), so the result is not an artifact of short-chain memorization.

Per-domain results are similar in pattern: vector *Contam-unsafe* ranges from 0.13 in research to 0.50 in finance and legal, with the three label-aware arms at 0.00 in every domain. The seven-domain spread is reported in the result file `results/benchmarks/product_comparison_results.json`; the structural conclusion — that the gap is uniform across domains — holds.

## 7.8 Negative control: single-shot RAG and PoisonedRAG

The rule constrains derived writeback. It does not stop a poisoned passage from being retrieved, does not robustly aggregate conflicting evidence, and does not classify raw passages by itself. To make that boundary explicit, we ran PoisonedRAG NQ [14] as a negative control: 100 single-shot QA questions with five attacker-crafted passages per question and two clean passages per question. Every passage was labeled by an LLM source classifier with no oracle supervision.

| Arm                             | Attack-success | Clean accuracy | Refusal | Tok/q |
|---------------------------------|----------------|----------------|---------|-------|
| <code>vector</code>             | 0.450          | 0.560          | 0.010   | 206   |
| <code>vector_with_labels</code> | 0.210          | 0.820          | 0.000   | 365   |
| <code>trace_memory</code>       | 0.220          | 0.820          | 0.000   | 393   |
| <code>bash</code>               | 0.270          | 0.680          | 0.000   | 1033  |
| <code>bash_nolabels</code>      | 0.300          | 0.650          | 0.000   | 956   |

Table 5: PoisonedRAG NQ negative-control results over 100 single-shot QA questions. The reduction from `vector` to `trace_memory` is not evidence for the source-downgrading inscription rule, because no derived record is written in this benchmark. The near-tie between `trace_memory` and `vector_with_labels` attributes the gain to the source classifier and label-aware prompt.

The classifier labeled 356 of 500 adversarial passages as low-trust (*fab*, *sim*, or *infer*) and mislabeled 144 as *external*; it labeled 196 of 200 clean passages as *external*. Thus the PoisonedRAG gain is a classifier-and-prompt result, not an inscription-rule result. PoisonedRAG is a single cycle of observe–retrieve–answer. There is no `add_derived` call between observation and answer, no derived record is inscribed, and Definition 4 is dormant by design. A deployable recursive-memory system needs both layers: source classification for raw ingestion and source downgrading for derivation chains.

## 7.9 Acceptance gates

The harness’s acceptance gates, encoded as pytest assertions:

- `source_downgrading` maintains zero on all four failure metrics across all  $N = 20$  seeds.
- `provenance_propagating` maintains zero local laundering and 1.0 provenance recall, but  $LR_{\text{truth}}$  remains strictly above zero.
- `naive_inscribe` maintains  $LR_{\text{truth}} = 1.0$  across all seeds.

All three gates pass.

## 8 Discussion

### 8.1 Implications for recursive self-indexing

The motivating goal is recursive self-indexing: a memory architecture in which the system’s records can point at the system’s own memory states, and those pointers themselves become first-class records [1]. Without a trust-composition rule, recursive self-indexing has a structural problem: each self-pointer is itself a derived record, and if derived records can acquire *external* trust by default (naive inscription), the recursion produces a fixed point in which the system’s beliefs about its own memory states have the same trust as external evidence. The system can then no longer distinguish what it observed from what it inferred about what it remembered observing.

Source downgrading prevents this collapse. Under Proposition 1, every step of recursive self-indexing produces a derived record with trust at most *infer*, and trust is monotonically non-increasing along the

recursion. If a self-pointer chains through a simulated or fabricated record, the trust degradation is even faster. The recursion is bounded: it cannot reach back up to external trust by any sequence of operations.

This is the property that makes recursive self-indexing safe enough to be useful. A recursion without trust degradation is self-amplifying; a recursion with monotone trust degradation is a calculus in which the system can reason about its own memory states without confusing them with the world.

## 8.2 API boundary

The rule is API-shaped: any `add_derived(content, contributing_record_ids)` method should compute the source label from contributing inputs rather than accepting a caller-supplied label. Caller-supplied labels are the laundering attack surface; computed labels close it. Retrieval, ranking, routing, and action selection may consult the computed label, but they should not silently relabel the derived record.

## 8.3 Limits of the present validation

The original fixture is deterministic and small. Five cases producing seven derived records is enough to exercise the relevant linear-chain input combinations as a conformance harness, but it is not a distribution over realistic inscription events. The 139-session adversarial-reload benchmark adds free-text LLM derivations across domains and chain depths one through three. Additional ablations would separate naive writeback with label-aware prompting, source-downgraded writeback hidden from the answer prompt, provenance propagation with labels visible, and routing-only uses of downgraded labels. The extended adversarial fixture of §7.6 additionally covers branching and reconvergent provenance DAGs, contradictory same-rank inputs, semantically irrelevant declared contributors, and one cross-agent label-stripping transport-boundary case. Remaining structural gaps include omitted contributors, imported provenance cycles, adversarial source-classifier prompt injection, long-chain over-suppression beyond depth three, and natural-trace calibration of the trust ordering. The cascade-invisibility property is demonstrated on a chain length of two; longer chains would amplify the local-versus-truth gap further but have not been exhaustively probed.

The deeper limit is that *Source* itself remains a partly-inferred function for natural content. Section 5 treats source labels as inputs to the inscription rule, not outputs of an inference. When labels are themselves inferred from content and retrieval features — as they must be for real agent operation [3] — the trust composition is no stronger than the weakest source-inference decision in the chain. The trust ordering is a sound rule given correct labels; building it on top of a label-inference layer that is reliable enough is a separate empirical program.

The present paper operates on hard labels. A conservative implementation can instead operate on lower-confidence source sets: if the classifier cannot rule out a lower-trust class, the inscription path should default downward rather than promote the record. This preserves the invariant’s spirit under uncertainty: source downgrading is sound conditional on labels, while source inference is empirical and fallible.

The present validation also fixes trust ranks globally. A deployed memory system may need context-sensitive trust: a stale external observation can be less useful than a recent signed tool result for one task, while the opposite may hold for another. This does not remove the need for source downgrading; it changes

the policy that supplies the active order. The conservative requirement is that context changes be explicit rather than accidental side effects of retrieval.

Cross-agent transfer is another boundary. If Agent A exports a *sim* record through an API and Agent B ingests it as ordinary tool output, the label can be lost before source downgrading has any chance to act. Multi-agent memory systems need signed provenance envelopes, source-label translation tables, or verified import policies. The rule in this paper constrains derived writes inside a labeled memory system; it does not secure label transport across system boundaries.

#### 8.4 What the rule does not do

Source downgrading is not a confidence calibration. The Trust function returns a rank, not a probability. Two derived records labeled *infer* are not equally well-justified; they are equally not-external. Combining the rule with calibrated confidence scoring is straightforward (and the harness already accepts a per-record `source_confidence` field), but the trust ordering and the confidence layer answer different questions.

The clean separation is:

$$\text{Source}(r) \neq \text{Credibility}_t(r) \neq \text{Confidence}_t(r).$$

Source says where the record came from or how it was produced. Source-integrity says whether that production chain is eligible for a given inscription or routing path. Credibility says how much the system currently believes the record’s content. Confidence says how reliable the system believes its own label, credibility estimate, or update operation to be. Source is monotone under derivation; credibility is revisable under evidence; confidence is estimated and calibrated.

This distinction prevents a common laundering variant. A discredited external observation remains externally sourced:

$$\text{Source}(r) = \text{ext}, \quad \text{Credibility}_t(r) \downarrow.$$

A confirmed inference remains an inference:

$$\text{Source}(r) = \text{infer}, \quad \text{Credibility}_t(r) \uparrow.$$

Promoting confirmed inferences to *external* would reintroduce laundering through a different field name. The correction-chain layer may raise or lower credibility; the source-downgrading layer prevents provenance inflation.

The rule also does not prevent over-suppression. An honest external observation that is composed with a simulated hypothesis produces a derived record at *sim* trust, and a downstream routing layer may quarantine it even when the simulation contribution was peripheral. Source downgrading errs on the side of trust-degradation; whether the degradation is warranted in any given case is a routing question, not an inscription one.

Finally, source downgrading is not a RAG-poisoning defence by itself. It does not filter malicious chunks at retrieval time, and it does not discover that an unlabeled passage is adversarial. PoisonedRAG shows the

boundary: a source classifier and label-aware prompt reduce single-shot attack success, but the inscription rule is inactive unless the system writes a derived record back into memory.

## 9 Future Work

The present paper deliberately fixes the trust ordering globally and assigns a single rank per source class. This is a methodological choice, not a theoretical limit: it lets us isolate the inscription invariant — derived trust cannot exceed input trust, with *infer* as the upper bound — and validate it before introducing further degrees of freedom. The invariant is what must be proven first, because every richer policy below assumes it. The extensions are listed in increasing order of how far they depart from the static lattice; each one preserves the integrity-meet rule of §5 and only enriches the valuation it operates on.

### 9.1 Per-record trust valuation

A natural first extension is to lift trust from a per-class rank to a per-record valuation. Let  $\tau : \mathcal{R} \rightarrow [0, 1]$  assign a scalar trust score to each record, subject to a class consistency constraint: the score for any record of class  $c$  must fall within an interval  $[\tau_c^-, \tau_c^+]$  such that the class intervals respect the ordering of §4. The inscription rule then becomes

$$\tau(\text{Inscribe}(\text{derived})) \leq \min(\tau(r_1), \dots, \tau(r_n), \tau_{\text{infer}}^+),$$

which collapses to the present rule when intervals degenerate to points. This captures within-class variance — two *ext* records from sensors of different known reliability, two *tool* outputs from a calibrated and an uncalibrated tool — without disturbing the lattice itself.

### 9.2 Context-dependent valuation

A deployed memory system encounters tasks in which the appropriate ordering shifts. A simulation-running planner treats *sim* as near-evidence; a perception agent treats it as near-fabrication. Generalizing  $\tau$  to  $\tau(r, \text{ctx})$ , where context is task-, role-, or query-conditioned, lets the active policy supply the order rather than hard-coding it. The conservative requirement, already stated in §8, is that every derivation be capped under whatever order is active when the derivation is inscribed, and that context transitions be explicit. The inscription invariant survives because the meet operates on  $(\text{class}, \tau)$  pairs in a context that is fixed at inscription time; only the valuation surface is context-relative.

### 9.3 Time-varying credibility

Belief revision can retract or update a record’s content when evidence arrives, but it should not rewrite how the record was produced. An *ext* observation discredited by later evidence retains its original source label even when its belief content is overturned. A symmetric gap exists in the other direction: an *infer* record

independently confirmed by multiple external arrivals could in principle be believed more strongly, but the meet rule of the present paper forbids promotion to *ext* in order to preserve laundering safety.

The clean resolution is to keep source labels monotone and let credibility vary independently. A discredited *ext* record stays *ext* in its class but acquires lower credibility. A confirmed *infer* record stays *infer* but may carry an annotated confirmation count or higher credibility that routing consults separately. This separates two distinct quantities the present paper intentionally keeps apart: provenance integrity (monotone, never promoted by derivation) and current credibility (time-varying, revised by evidence).

## 9.4 Label transport and cross-agent provenance

Section 8 already flags the cross-agent boundary: labels are easy to lose at API edges. A future-work item is to specify signed provenance envelopes and verified import policies that preserve source labels across agent or service boundaries, including translation rules when class taxonomies differ. The inscription rule of this paper presupposes labeled inputs; closing the transport gap is what makes that presupposition robust in multi-agent deployments.

**Order of work.** The static-lattice invariant has now been validated on the deterministic conformance fixture, the embedding-noise sweep, and the 139-session adversarial-reload benchmark. The next step is the per-record valuation extension on a synthetic fixture with known sensor reliabilities, followed by explicit time-varying credibility and confidence layers. Context-dependent ordering and signed cross-agent transport are downstream of both. Each step preserves the meet rule of §5; the foundation is what makes the rest expressible.

## 10 Conclusion

A memory architecture that derives new records from old records must constrain how trust composes across derivation. The constraint is a standard integrity-meet rule from lattice-based information-flow control, adapted here to source labels for recursive agent memory: derived trust cannot exceed input trust, and the act of derivation itself imposes *infer* as the upper bound. We have defined the source-lattice policy, defended the default ordering operationally, validated the rule on a deterministic conformance fixture across five cases and twenty seeds, evaluated the full source-aware path on a 139-session LLM-in-the-loop benchmark, and identified one methodological constraint — cascade invisibility — that any audit of laundering safety must respect.

The rule is small, falsifiable, and composable. It does not require a new architecture; it constrains what an existing architecture’s inscription path must compute. Its scope is equally narrow: it is a writeback safety rule for derived records, not a retrieval-poisoning defence or hallucination detector. Without the rule, recursive self-indexing produces self-amplifying memory; with the rule, it produces memory in which the system can compose operations over its own outputs without confusing them with the world.



## Code Availability

The artifact is consolidated in this repository and is intended to be archived with a versioned release corresponding to this manuscript. The deterministic conformance harness, three-policy comparison, multi-seed sweep, and pytest acceptance gates are implemented under `src/fgm/`, `tests/architecture/`, and `results/architecture/`. The larger adversarial-reload v2 and PoisonedRAG negative-control benchmarks are implemented under `benchmarks/`. Result files are committed as:

- `results/architecture/laundrying_validation_summary.json`
- `results/architecture/laundrying_validation_multiseed_summary.json`
- `results/benchmarks/product_comparison_results.json`
- `results/benchmarks/PRODUCT_COMPARISON.md`
- `results/benchmarks/poisonedrag_results.json`
- `results/benchmarks/POISONEDRAG.md`

The live/API result files are treated as committed evidence for this manuscript; the reproducibility notes distinguish them from non-live tests and deterministic reruns.

## Funding and Competing Interests

The author is the founder of Tyxter. No external funding supported this work. The author declares no competing interests.

## References

- [1] D. F. Carboni. Self-Indexed Mnestic Fold Calculus: A Graph-Operator Framework for Recursive Memory and Emergent Self-Indexing. Zenodo, 2025. doi:10.5281/zenodo.20042034.
- [2] D. F. Carboni. Attention Is Not Continuity: Mnestic Continuity as a Trace-Theoretic Substrate for Autonomous LLM Agents. Zenodo, 2026. doi:10.5281/zenodo.20057986.
- [3] D. F. Carboni. Source Inference and Inference Laundering: Validation Results. Technical note, docs/architecture/SOURCE\_INFERENCE\_AND\_LAUNDERING.md, 2026.
- [4] D. E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi:10.1145/360051.360056.
- [5] A. C. Myers and B. Liskov. A decentralized model for information flow control. In *Proceedings of the 16th ACM Symposium on Operating Systems Principles*, pages 129–142, 1997. doi:10.1145/268998.266669.
- [6] A. Sabelfeld and A. C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi:10.1109/JSAC.2002.806121.

- [7] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, pages 31–40, 2007. doi:10.1145/1265530.1265535.
- [8] J. Cheney, L. Chiticariu, and W.-C. Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. doi:10.1561/1900000006.
- [9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33, 2020.
- [10] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560, 2023.
- [11] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.
- [12] M. K. Johnson, S. Hashtroudi, and S. D. Lindsay. Source Monitoring. *Psychological Bulletin*, 114(1):3–28, 1993.
- [13] M. K. Johnson and C. L. Raye. Reality Monitoring. *Psychological Review*, 88(1):67–85, 1981.
- [14] W. Zou, R. Geng, B. Wang, and J. Jia. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *34th USENIX Security Symposium*, pages 3827–3844, 2025.